AIAC LAB 9.1

Lab Experiment: Documentation Generation – Automatic documentation and code comments

Name :U.Pranav Dev
Roll No : 2503A52694

Batch No : 24

Problem 1 – Documenting find_max() in Three Styles

Question:
Consider the following Python function:
def find_max(numbers):
    return max(numbers)
Task: Write documentation for the function in all three formats: (a) Docstring (b) Inline comments (c) Google-style documentation. Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style. Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Answer:
The Python program below documents find_max() in all three formats and takes the list of numbers from the user.

#code

problem1.py • Python

```python
import inspect


# (a) Docstring version
def find_max_docstring(numbers):
    """Return the largest number in a list.

    Takes a list of numbers and returns the maximum value
    using Python's built-in max() function.
    """
    return max(numbers)


# (b) Inline comments version
def find_max_inline(numbers):
    # Use the built-in max() to scan the whole list
    # and pick the largest element
    return max(numbers)  # returns the maximum value


```

problem1.py • Python

```python
# (c) Google-style documentation version
def find_max_google(numbers):
    """Finds the maximum value in a list of numbers.

    Args:
        numbers (list): A non-empty list of int or float values.

    Returns:
        int or float: The largest value in the list.

    Raises:
        ValueError: If the list is empty.

    Example:
        >>> find_max_google([3, 7, 2])
        7
    """
    return max(numbers)


# User input
values = input("Enter integers separated by spaces: ")
numbers = [int(x) for x in values.split()]

print("Docstring version     :", find_max_docstring(numbers))
print("Inline comment version:", find_max_inline(numbers))
print("Google style version  :", find_max_google(numbers))

print("\nDocstring of (a):")
print(inspect.getdoc(find_max_docstring))
print("\nDocstring of (b):")
print(inspect.getdoc(find_max_inline))
print("\nDocstring of (c):")
print(inspect.getdoc(find_max_google))
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python problem1.py
Enter integers separated by spaces: 45 12 89 33 7
Docstring version     : 89
Inline comment version: 89
Google style version  : 89

Docstring of (a):
Return the largest number in a list.

Takes a list of numbers and returns the maximum value
using Python's built-in max() function.

Docstring of (b):
None

Docstring of (c):
Finds the maximum value in a list of numbers.

Args:
    numbers (list): A non-empty list of int or float values.

Returns:
    int or float: The largest value in the list.

Raises:
    ValueError: If the list is empty.

Example:
    >>> find_max_google([3, 7, 2])
    7
```

#comparison of the three styles

Style	Advantages	Disadvantages	Suitable use cases
(a) Docstring	Stored with the function, so help(), pydoc and IDE tooltips can show it at run time. Short and standard (PEP 257).	A plain summary has no fixed structure, so parameter, return and error details are easy to leave out.	Small, simple functions that only need a short description.
(b) Inline comments	Sit next to the exact line they explain; good for explaining why something is done.	Not available through help() or documentation tools; easily become outdated; a comment on a one-line max() call adds clutter.	Tricky or non-obvious logic inside a function body, mainly for maintainers reading the source.
(c) Google-style	Clear Args / Returns / Raises / Example sections; easy to read as plain text; Sphinx (napoleon) and IDEs can parse it; examples can run as doctests.	Longer than the code it describes for tiny functions; must be updated whenever behaviour changes.	Public APIs, libraries and team projects where other people call the function without reading its code.

Recommendation: Google-style docstrings are the most effective for a mathematical utilities library. Mathematical functions need precise contracts: the expected input types, what happens in edge cases (an empty list raises ValueError), the return type and a worked example. The Args / Returns / Raises / Example sections capture all of this in a fixed structure, the examples double as doctests, and tools such as help(), pydoc, IDE tooltips and Sphinx can turn them into reference pages for the library's users. Inline comments should only supplement them where a line of logic is not obvious.

Problem 2 – Documenting login()

Question:
Consider the following Python function:
def login(user, password, credentials):
    return credentials.get(user) == password
Task: Write documentation in all three formats. Critically compare the approaches. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Answer:
The Python program below documents login() in all three formats and takes the username and password from the user.

#code

problem2.py • Python

```python
# Demo credentials (username -> password)
credentials = {"admin": "Admin@123", "lakshitha": "Pass@2710", "guest": "guest123"}


# (a) Docstring version
def login_docstring(user, password, credentials):
    """Check whether the username and password are valid.

    Looks up the user in the credentials dictionary and compares
    the stored password with the password entered.
    """
    return credentials.get(user) == password


# (b) Inline comments version
def login_inline(user, password, credentials):
    # get() returns None if the user does not exist, so unknown users fail
    # Compare the stored password with the entered one (True = login success)
    return credentials.get(user) == password


```

problem2.py • Python

```python
# (c) Google-style documentation version
def login_google(user, password, credentials):
    """Validates a user's login attempt.

    Args:
        user (str): Username entered by the user.
        password (str): Password entered by the user.
        credentials (dict): Mapping of username (str) to password (str).

    Returns:
        bool: True if the user exists and the password matches,
            otherwise False.

    Note:
        Passwords are compared as plain text (demo only). Real systems
        should store and compare hashed passwords.

    Example:
        >>> login_google("admin", "Admin@123", {"admin": "Admin@123"})
        True
    """
    return credentials.get(user) == password


# User input
user = input("Enter username: ")
password = input("Enter password: ")

print("Docstring version :", login_docstring(user, password, credentials))
print("Inline version    :", login_inline(user, password, credentials))
print("Google version    :", login_google(user, password, credentials))

if login_google(user, password, credentials):
    print("Login successful")
else:
    print("Login failed")
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python problem2.py
Enter username: lakshitha
Enter password: Pass@2710
Docstring version : True
Inline version    : True
Google version    : True
Login successful
PS C:\Users\V\OneDrive\AIAC> python problem2.py
Enter username: admin
Enter password: wrong123
Docstring version : False
Inline version    : False
Google version    : False
Login failed
```

#comparison of the three styles

Style	Advantages	Disadvantages	Suitable use cases
(a) Docstring	Explains the purpose of login() in help(login) with very little effort.	A short summary does not say what shape credentials must have or what is returned for an unknown user unless the writer adds it.	Quick description of small helper functions.

Style	Advantages	Disadvantages	Suitable use cases
(b) Inline comments	Explain details such as get() returning None for unknown users exactly where the logic sits.	A newcomer only sees them after opening the source file; not shown by help() or generated documentation.	Clarifying tricky lines for people already working inside the file.
(c) Google-style	Args, Returns, Note and Example tell a developer what to pass in (including the credentials dictionary format) and what to expect, without reading the body. Same layout across the project.	Takes longer to write and must be kept in sync with the code.	Shared or public functions in a team codebase, especially for onboarding.

Recommendation: Google-style docstrings are the most helpful for new developers onboarding a project. A newcomer can call login() correctly just by reading its docstring: it shows the type of every argument, the structure of the credentials dictionary, that the result is a bool (False for an unknown user), a runnable example and a security note about plain-text passwords. The same text appears in help(), IDE tooltips and generated HTML documentation, so nobody has to open the source. Inline comments are a good supplement for one specific line, but they are invisible to documentation tools.

Problem 3 – Calculator (Automatic Documentation Generation)

Question:
Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.
Instructions: Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
add(a, b) – returns the sum of two numbers
subtract(a, b) – returns the difference of two numbers
multiply(a, b) – returns the product of two numbers
divide(a, b) – returns the quotient of two numbers
Display the module documentation in the terminal using Python's documentation tools. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

Answer:
The module calculator.py below contains add(), subtract(), multiply() and divide(), each with a docstring. A main block takes the numbers and the operation from the user when the file is run directly.

#code

calculator.py • Python

```python
"""Calculator module.

This module provides basic arithmetic operations: addition,
subtraction, multiplication and division.
"""


def add(a, b):
    """Return the sum of two numbers.

    Args:
        a: First number.
        b: Second number.

    Returns:
        The sum of a and b.
    """
    return a + b


def subtract(a, b):
    """Return the difference of two numbers.

    Args:
        a: Number to subtract from.
        b: Number to subtract.

    Returns:
        The result of a - b.
    """
    return a - b


def multiply(a, b):
    """Return the product of two numbers.

    Args:
        a: First number.
        b: Second number.

    Returns:
        The product of a and b.
    """
    return a * b


```

calculator.py • Python

```python
def divide(a, b):
    """Return the quotient of two numbers.

    Args:
        a: Dividend (number to be divided).
        b: Divisor (number to divide by).

    Returns:
        The result of a / b.

    Raises:
        ZeroDivisionError: If b is zero.
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b


if __name__ == "__main__":
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))
    op = input("Enter operation (+, -, *, /): ")

    try:
        if op == "+":
            result = add(a, b)
        elif op == "-":
            result = subtract(a, b)
        elif op == "*":
            result = multiply(a, b)
        elif op == "/":
            result = divide(a, b)
        else:
            raise ValueError("Invalid operation")
        print(f"Result: {a:g} {op} {b:g} = {result:g}")
    except (ValueError, ZeroDivisionError) as error:
        print("Error:", error)
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python calculator.py
Enter first number: 15
Enter second number: 3
Enter operation (+, -, *, /): /
Result: 15 / 3 = 5
PS C:\Users\V\OneDrive\AIAC> python calculator.py
Enter first number: 10
Enter second number: 0
Enter operation (+, -, *, /): /
Error: Cannot divide by zero
PS C:\Users\V\OneDrive\AIAC> python calculator.py
Enter first number: 7
Enter second number: 5
Enter operation (+, -, *, /): +
Result: 7 + 5 = 12
```

#documentation in the terminal (python -m pydoc calculator)

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python -m pydoc calculator
Help on module calculator:

NAME
    calculator - Calculator module.

DESCRIPTION
    This module provides basic arithmetic operations: addition,
    subtraction, multiplication and division.

FUNCTIONS
    add(a, b)
        Return the sum of two numbers.

        Args:
            a: First number.
            b: Second number.

        Returns:
            The sum of a and b.

    divide(a, b)
        Return the quotient of two numbers.

        Args:
            a: Dividend (number to be divided).
            b: Divisor (number to divide by).

        Returns:
            The result of a / b.

        Raises:
            ZeroDivisionError: If b is zero.
```

```
TERMINAL
    multiply(a, b)
        Return the product of two numbers.

        Args:
            a: First number.
            b: Second number.

        Returns:
            The product of a and b.

    subtract(a, b)
        Return the difference of two numbers.

        Args:
            a: Number to subtract from.
            b: Number to subtract.

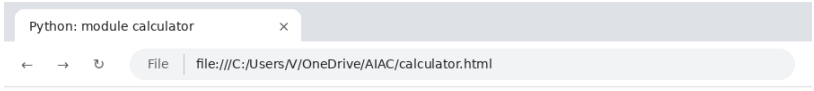
        Returns:
            The result of a - b.

FILE
    C:\Users\V\OneDrive\AIAC\calculator.py
```

#export documentation as HTML (python -m pydoc -w calculator)

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python -m pydoc -w calculator
wrote calculator.html
```

#calculator.html opened in a web browser



index
calculator C:\Users\V\OneDrive\AIAC\calculator.py

Calculator module.

This module provides basic arithmetic operations: addition,
subtraction, multiplication and division.

Functions

add(a, b)
Return the sum of two numbers.

Args:
a: First number.
b: Second number.

Returns:
The sum of a and b.

divide(a, b)
Return the quotient of two numbers.

Args:
a: Dividend (number to be divided).
b: Divisor (number to divide by).

Returns:
The result of a / b.

Raises:
ZeroDivisionError: If b is zero.

multiply(a, b)
Return the product of two numbers.

Args:
a: First number.
b: Second number.

Returns:
The product of a and b.

subtract(a, b)
Return the difference of two numbers.

Args:
a: Number to subtract from.
b: Number to subtract.

Returns:
The result of a - b.

Problem 4 – Conversion Utilities Module

Question:
Task: Write a module named conversion.py with functions: decimal_to_binary(n), binary_to_decimal(b), decimal_to_hexadecimal(n).
Use Copilot for auto-generating docstrings. Generate documentation in the terminal. Export the documentation in HTML format and open it in a browser.

Answer:
The module conversion.py below contains the three conversion functions, each with a docstring. A main block asks the user which conversion to perform and reads the number to convert.

#code

conversion.py • Python

```python
"""Conversion utilities module.

This module provides functions to convert numbers between the
decimal, binary and hexadecimal number systems.
"""


def decimal_to_binary(n):
    """Convert a decimal integer to a binary string.

    Args:
        n (int): A non-negative decimal integer.

    Returns:
        str: Binary representation of n (without the '0b' prefix).

    Raises:
        ValueError: If n is negative.

    Example:
        >>> decimal_to_binary(10)
        '1010'
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")
    return bin(n)[2:]


def binary_to_decimal(b):
    """Convert a binary string to a decimal integer.

    Args:
        b (str): A string made up of only the digits 0 and 1.

    Returns:
        int: Decimal value of the binary number.

    Raises:
        ValueError: If b contains characters other than 0 and 1.

    Example:
        >>> binary_to_decimal("1010")
        10
    """
    if b == "" or any(c not in "01" for c in b):
        raise ValueError("b must contain only 0 and 1")
    return int(b, 2)


```

conversion.py • Python

```python
def decimal_to_hexadecimal(n):
    """Convert a decimal integer to a hexadecimal string.

    Args:
        n (int): A non-negative decimal integer.

    Returns:
        str: Uppercase hexadecimal representation of n
            (without the '0x' prefix).

    Raises:
        ValueError: If n is negative.

    Example:
        >>> decimal_to_hexadecimal(255)
        'FF'
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")
    return hex(n)[2:].upper()


if __name__ == "__main__":
    print("1. Decimal to Binary  2. Binary to Decimal  3. Decimal to Hexadecimal")
    choice = input("Enter your choice: ")

    try:
        if choice == "1":
            n = int(input("Enter a decimal number: "))
            print("Binary:", decimal_to_binary(n))
        elif choice == "2":
            b = input("Enter a binary number: ")
            print("Decimal:", binary_to_decimal(b))
        elif choice == "3":
            n = int(input("Enter a decimal number: "))
            print("Hexadecimal:", decimal_to_hexadecimal(n))
        else:
            print("Invalid choice")
    except ValueError as error:
        print("Error:", error)
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python conversion.py
1. Decimal to Binary  2. Binary to Decimal  3. Decimal to Hexadecimal
Enter your choice: 1
Enter a decimal number: 10
Binary: 1010
PS C:\Users\V\OneDrive\AIAC> python conversion.py
1. Decimal to Binary  2. Binary to Decimal  3. Decimal to Hexadecimal
Enter your choice: 2
Enter a binary number: 1010
Decimal: 10
PS C:\Users\V\OneDrive\AIAC> python conversion.py
1. Decimal to Binary  2. Binary to Decimal  3. Decimal to Hexadecimal
Enter your choice: 3
Enter a decimal number: 255
Hexadecimal: FF
PS C:\Users\V\OneDrive\AIAC> python conversion.py
1. Decimal to Binary  2. Binary to Decimal  3. Decimal to Hexadecimal
Enter your choice: 2
Enter a binary number: 1021
Error: b must contain only 0 and 1
```

#documentation in the terminal (python -m pydoc conversion)

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python -m pydoc conversion
Help on module conversion:

NAME
    conversion - Conversion utilities module.

DESCRIPTION
    This module provides functions to convert numbers between the
    decimal, binary and hexadecimal number systems.

FUNCTIONS
    binary_to_decimal(b)
        Convert a binary string to a decimal integer.

        Args:
            b (str): A string made up of only the digits 0 and 1.

        Returns:
            int: Decimal value of the binary number.

        Raises:
            ValueError: If b contains characters other than 0 and 1.

        Example:
            >>> binary_to_decimal("1010")
            10
```

```
TERMINAL
    decimal_to_binary(n)
        Convert a decimal integer to a binary string.

        Args:
            n (int): A non-negative decimal integer.

        Returns:
            str: Binary representation of n (without the '0b' prefix).

        Raises:
            ValueError: If n is negative.

        Example:
            >>> decimal_to_binary(10)
            '1010'

    decimal_to_hexadecimal(n)
        Convert a decimal integer to a hexadecimal string.

        Args:
            n (int): A non-negative decimal integer.

        Returns:
            str: Uppercase hexadecimal representation of n
                (without the '0x' prefix).

        Raises:
            ValueError: If n is negative.

        Example:
            >>> decimal_to_hexadecimal(255)
            'FF'

FILE
    C:\Users\V\OneDrive\AIAC\conversion.py
```

#export documentation as HTML (python -m pydoc -w conversion)

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python -m pydoc -w conversion
wrote conversion.html
```

#conversion.html opened in a web browser



index
conversion C:\Users\V\OneDrive\AIAC\conversion.py

Conversion utilities module.

This module provides functions to convert numbers between the
decimal, binary and hexadecimal number systems.

Functions

binary_to_decimal(b)
Convert a binary string to a decimal integer.

Args:
b (str): A string made up of only the digits 0 and 1.

Returns:
int: Decimal value of the binary number.

Raises:
ValueError: If b contains characters other than 0 and 1.

Example:
>>> binary_to_decimal("1010")
10

decimal_to_binary(n)
Convert a decimal integer to a binary string.

Args:
n (int): A non-negative decimal integer.

Returns:
str: Binary representation of n (without the '0b' prefix).

Raises:
ValueError: If n is negative.

Example:
>>> decimal_to_binary(10)
'1010'

decimal_to_hexadecimal(n)
Convert a decimal integer to a hexadecimal string.

Args:
n (int): A non-negative decimal integer.

Returns:
str: Uppercase hexadecimal representation of n
(without the '0x' prefix).

Raises:
ValueError: If n is negative.

Example:
>>> decimal_to_hexadecimal(255)
'FF'

Problem 5 – Course Management Module

Question:
Task: Create a module course.py with functions: add_course(course_id, name, credits), remove_course(course_id), get_course(course_id).
Add docstrings with Copilot. Generate documentation in the terminal. Export the documentation in HTML format and open it in a browser.

Answer:
The module course.py below stores courses in memory and provides add_course(), remove_course() and get_course(), each with a docstring. A menu-driven main block takes the course details from the user.

#code

course.py • Python

```python
"""Course management module.

This module stores courses in memory and provides functions to
add, remove and look up courses by their course ID.
"""

courses = {}


def add_course(course_id, name, credits):
    """Add a new course to the course list.

    Args:
        course_id (str): Unique ID of the course, e.g. "CS101".
        name (str): Name of the course.
        credits (int): Number of credits for the course.

    Returns:
        bool: True if the course was added, False if the
            course_id already exists.
    """
    if course_id in courses:
        return False
    courses[course_id] = {"name": name, "credits": credits}
    return True


def remove_course(course_id):
    """Remove a course from the course list.

    Args:
        course_id (str): ID of the course to remove.

    Returns:
        bool: True if the course was removed, False if it
            does not exist.
    """
    if course_id not in courses:
        return False
    del courses[course_id]
    return True


```

course.py • Python

```python
def get_course(course_id):
    """Get the details of a course.

    Args:
        course_id (str): ID of the course to look up.

    Returns:
        dict or None: A dictionary with the keys "name" and
            "credits", or None if the course does not exist.
    """
    return courses.get(course_id)


if __name__ == "__main__":
    while True:
        print("\n1. Add course  2. Remove course  3. Get course  4. Exit")
        choice = input("Enter your choice: ")

        if choice == "1":
            course_id = input("Enter course ID: ")
            name = input("Enter course name: ")
            credits = int(input("Enter credits: "))
            if add_course(course_id, name, credits):
                print("Course added")
            else:
                print("Course ID already exists")
        elif choice == "2":
            course_id = input("Enter course ID: ")
            if remove_course(course_id):
                print("Course removed")
            else:
                print("Course not found")
        elif choice == "3":
            course_id = input("Enter course ID: ")
            course = get_course(course_id)
            if course:
                print("Name:", course["name"], "| Credits:", course["credits"])
            else:
                print("Course not found")
        elif choice == "4":
            print("Exiting course manager")
            break
        else:
            print("Invalid choice")
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python course.py

1. Add course  2. Remove course  3. Get course  4. Exit
Enter your choice: 1
Enter course ID: CS101
Enter course name: Python Programming
Enter credits: 4
Course added

1. Add course  2. Remove course  3. Get course  4. Exit
Enter your choice: 1
Enter course ID: MA201
Enter course name: Discrete Mathematics
Enter credits: 3
Course added

1. Add course  2. Remove course  3. Get course  4. Exit
Enter your choice: 3
Enter course ID: CS101
Name: Python Programming | Credits: 4

1. Add course  2. Remove course  3. Get course  4. Exit
Enter your choice: 2
Enter course ID: MA201
Course removed

1. Add course  2. Remove course  3. Get course  4. Exit
Enter your choice: 3
Enter course ID: MA201
Course not found

1. Add course  2. Remove course  3. Get course  4. Exit
Enter your choice: 4
Exiting course manager
```

#documentation in the terminal (python -m pydoc course)

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python -m pydoc course
Help on module course:

NAME
    course - Course management module.

DESCRIPTION
    This module stores courses in memory and provides functions to
    add, remove and look up courses by their course ID.

FUNCTIONS
    add_course(course_id, name, credits)
        Add a new course to the course list.

        Args:
            course_id (str): Unique ID of the course, e.g. "CS101".
            name (str): Name of the course.
            credits (int): Number of credits for the course.

        Returns:
            bool: True if the course was added, False if the
                course_id already exists.
```

```
TERMINAL
    get_course(course_id)
        Get the details of a course.

        Args:
            course_id (str): ID of the course to look up.

        Returns:
            dict or None: A dictionary with the keys "name" and
                "credits", or None if the course does not exist.

    remove_course(course_id)
        Remove a course from the course list.

        Args:
            course_id (str): ID of the course to remove.

        Returns:
            bool: True if the course was removed, False if it
                does not exist.

DATA
    courses = {}

FILE
    C:\Users\V\OneDrive\AIAC\course.py
```

#export documentation as HTML (python -m pydoc -w course)

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python -m pydoc -w course
wrote course.html
```

#course.html opened in a web browser



index
course C:\Users\V\OneDrive\AIAC\course.py

Course management module.

This module stores courses in memory and provides functions to
add, remove and look up courses by their course ID.

Functions

add_course(course_id, name, credits)
Add a new course to the course list.

Args:
course_id (str): Unique ID of the course, e.g. "CS101".
name (str): Name of the course.
credits (int): Number of credits for the course.

Returns:
bool: True if the course was added, False if the
course_id already exists.

get_course(course_id)
Get the details of a course.

Args:
course_id (str): ID of the course to look up.

Returns:
dict or None: A dictionary with the keys "name" and
"credits", or None if the course does not exist.

remove_course(course_id)
Remove a course from the course list.

Args:
course_id (str): ID of the course to remove.

Returns:
bool: True if the course was removed, False if it
does not exist.

Data

courses = {}

Conclusion

All five problems were completed. find_max() and login() were documented using docstrings, inline comments and Google-style documentation and the styles were compared. The modules calculator.py, conversion.py and course.py were documented with docstrings, viewed in the terminal with pydoc, exported as HTML pages with pydoc -w and verified in a web browser, as required in Lab 9.1.